

ARIN5204: Reinforcement Learning

2.4 Advanced Tricks for DQNs

Dr. Long Chen

longchen@ust.hk

Assistant Professor, Dept. of CSE

The Hong Kong University of Science and Technology (HKUST)

March 23, 2026



- 1 Experience Replay
- 2 Prioritized ER
- 3 Target Network & Double DQN
- 4 Dueling Network

Some slides are from: Sergey Levine (UCB), Shusen Wang (Meta), Katerina Fragkiadaki (CMU).

① Experience Replay

② Prioritized ER

③ Target Network & Double DQN

④ Dueling Network

Deep Q-Learning [1]

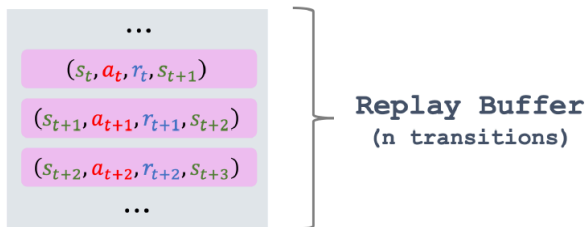
- To alleviate the problems of **correlated data** and **non-stationary distributions**
- Proposed experience replay [2]
- Results: outperformed **all previous RL algorithms** on **six of the seven** games

[1] Playing Atari with Deep Reinforcement Learning. In NIPS workshop, 2013.

[2] Reinforcement Learning for Robots using Neural Networks. Technical Report, 1993.

Experience Replay

- A transition: (s_t, a_t, r_t, s_{t+1})
- Store recent n transitions in a replay buffer.



-
- [1] A deeper look at experience replay. In NeurIPS workshop, 2017.
[2] Revisiting fundamentals of experience replay. In ICML, 2019.

Experience Replay

- A transition: (s_t, a_t, r_t, s_{t+1})
- Store recent n transitions in a replay buffer.
- Remove old transitions so that the buffer has at most n transitions.
- **Buffer capacity** n is a tuning hyper-parameter.
 - n is typically large, e.g., $10^5 \sim 10^6$
 - The setting of n is application-specific

[1] A deeper look at experience replay. In NeurIPS workshop, 2017.

[2] Revisiting fundamentals of experience replay. In ICML, 2019.

TD with Experience Replay

- Find $\mathbf{w}$ by minimizing $L(\mathbf{w}) = \frac{1}{T} \sum_{t=1}^T \frac{\delta_t^2}{2}$
- Stochastic gradient descent (SGD)
 - Randomly sample a transition, (s_i, a_i, r_i, s_{i+1}) , from the buffer
 - Compute TD errors, δ_i
 - Stochastic gradient: $\mathbf{g}_i = \frac{\partial \delta_i^2 / 2}{\partial \mathbf{w}} = \delta_i \cdot \frac{\partial Q(s_i, a_i; \mathbf{w})}{\partial \mathbf{w}}$
 - SGD: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \mathbf{g}_i$

Advantages of Experience Replay (ER)

- Each step of experience is potentially used in **many weight updates**, which **allows for greater data efficiency**.
- Learning directly from consecutive samples is inefficient, due to the **strong correlations between the samples**.
- Instead, randomizing the samples breaks these correlations and therefore **reduces the variance of the updates**.
- By using **experience replay**, the behavior distribution is averaged over many of its previous states, smoothing out learning and avoiding oscillations or divergence in the parameters.

Playing Atari with Deep Reinforcement Learning. In NIPS workshop, 2013.

Algorithm: Deep Q-Learning

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N

Initialize action-value function Q with random weights

for episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$

for $t = 1, T$ **do**

 With probability ϵ select a random action a_t

 otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$

 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3

end for

end for

Playing Atari with Deep Reinforcement Learning. In NIPS workshop, 2013.

- 1 Experience Replay
- 2 Prioritized ER
- 3 Target Network & Double DQN
- 4 Dueling Network

Prioritized Experience Replay

- **Basic Idea:** not all transitions are equally important
- Q: Which kind of transitions is more important, left or right?

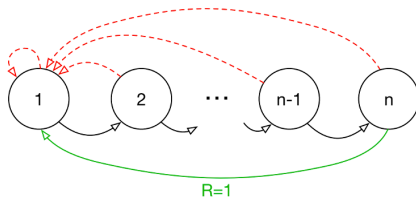


Prioritized Experience Replay. In ICLR, 2016

Prioritized Experience Replay

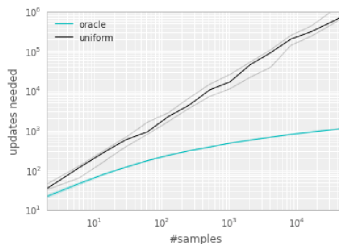
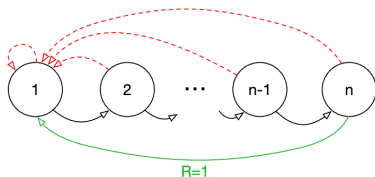
A Motivative Example (Blind Cliffwalk)

- With only n states, the environment requires an exponential number of random steps until the first non-zero reward
- To be precise, the chance that a random sequence of actions will lead to the reward is 2^{-n} .
- **Dashed red arrow:** The episode is terminated whenever the agent takes the “wrong” action



Prioritized Experience Replay. In ICLR, 2016

Prioritized Experience Replay



- Median number of learning steps required to learn the value function as a function of the size of the total number of transitions in the replay memory.
- Note the log-log scale, which highlights the exponential speed-up from replaying with an oracle (bright blue), compared to uniform replay (black).

Prioritized Experience Replay. In ICLR, 2016

Recap: Asynchronous Dynamic Programming

Prioritized Sweeping (Lecture 1.4)

- Use the **magnitude of Bellman error** to guide state selection, e.g.,

$$\left| \max_a \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V(s') \right) - V(s) \right|$$

- Backup the state with the largest remaining Bellman error
- Can be implemented efficiently by maintaining a priority queue
- Prioritized Experience Replay**: Prioritized sweeping (Moore & Atkeson, 1993; Andre *et al.*, 1998) selects which state to update next, prioritized according to the change in value, if that update was executed.

Prioritized Experience Replay. In ICLR, 2016

Prioritized Experience Replay

- **Q: How to measure the importance of each transition?**
- **Simplest way:** the magnitude of a transitions TD error δ
- It is particularly suitable for incremental, online RL algorithms (SARSA, Q-Learning)

$$p(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$$

where $p_i > 0$ is the priority of transition i .

- The exponent α determines how much prioritization is used.
- Especially, $\alpha = 0$ corresponding to the uniform case.

Prioritized Experience Replay. In ICLR, 2016

Prioritized Experience Replay

- Motivation: big $|\delta_t|$ shall be given high priority.
- Option 1: Sampling probability $p_i \propto |\delta_i| + \epsilon$
- Option 2: Sampling probability $p_i \propto \frac{1}{rank(i)}$
 - The transitions are sorted so that $|\delta_t|$ is in the descending order
 - $rank(i)$ is the rank of the i -th transition.

Update TD Error

- Associate each transition, (s_t, a_t, r_t, s_{t+1}) , with a TD error, δ_t
- If a transition is newly collected, we do not know its δ_t
 - Simple set its δ_t to the maximum
 - It has the highest priority
- Each time (s_t, a_t, r_t, s_{t+1}) is selected from the buffer, we update its δ_t

Scaling Learning Rate

- SGD: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \mathbf{g}$, where α is the learning rate
- If **uniform sampling** is used, α is the same for all transitions
- If **weighted sampling** is used, α shall be adjusted according to the importance
- **Q: why?**
- Scale the learning rate by $(np_i)^{-\beta}$, where $\beta \in (0, 1)$.
 - If $p_1 = p_2 = \dots = p_n = \frac{1}{n}$ (uniform sampling), the scaling factor is equal to 1.
 - High-importance transitions (with high p_i) have low learning rates
 - In the beginning, set β small; increase β to 1 over time.

Prioritized Experience Replay. In ICLR, 2016

Possible Question

- Q: If transition s_t, a_t, r_t, s_{t+1} is important, we increase the sampling weight to 10 times, but we decrease the learning rate to $1/10$. Does the prioritized experience replay still work?
- Case 1: learning rate α , use sample s_t, a_t, r_t, s_{t+1} to calculate 1 time gradient, and update 1 time parameter
- Case 2: learning rate $\alpha/10$, use sample s_t, a_t, r_t, s_{t+1} to calculate 10 times gradient, and update 10 times parameter
- : Which one is better?
- A: Case 2 usually works better, but its computation is 10 times.

Prioritized Experience Replay

Transitions	Sampling Probabilities	Learning Rates
...	...	...
$(s_t, a_t, r_t, s_{t+1}), \delta_t$	$p_t \propto \delta_t + \epsilon$	$\alpha \cdot (n p_t)^{-\beta}$
$(s_{t+1}, a_{t+1}, r_{t+1}, s_{t+2}), \delta_{t+1}$	$p_{t+1} \propto \delta_{t+1} + \epsilon$	$\alpha \cdot (n p_{t+1})^{-\beta}$
$(s_{t+2}, a_{t+2}, r_{t+2}, s_{t+3}), \delta_{t+2}$	$p_{t+2} \propto \delta_{t+2} + \epsilon$	$\alpha \cdot (n p_{t+2})^{-\beta}$
...	...	...

Summary: Big $|\delta_t| \Rightarrow$ High probability $\Rightarrow$ Small learning rate

Prioritized Experience Replay

Algorithm 1 Double DQN with proportional prioritization

```
1: Input: minibatch  $k$ , step-size  $\eta$ , replay period  $K$  and size  $N$ , exponents  $\alpha$  and  $\beta$ , budget  $T$ .
2: Initialize replay memory  $\mathcal{H} = \emptyset$ ,  $\Delta = 0$ ,  $p_1 = 1$ 
3: Observe  $S_0$  and choose  $A_0 \sim \pi_\theta(S_0)$ 
4: for  $t = 1$  to  $T$  do
5:   Observe  $S_t, R_t, \gamma_t$ 
6:   Store transition  $(S_{t-1}, A_{t-1}, R_t, \gamma_t, S_t)$  in  $\mathcal{H}$  with maximal priority  $p_t = \max_{i < t} p_i$ 
7:   if  $t \equiv 0 \pmod K$  then
8:     for  $j = 1$  to  $k$  do
9:       Sample transition  $j \sim P(j) = p_j^\alpha / \sum_i p_i^\alpha$ 
10:      Compute importance-sampling weight  $w_j = (N \cdot P(j))^{-\beta} / \max_i w_i$ 
11:      Compute TD-error  $\delta_j = R_j + \gamma_j Q_{\text{target}}(S_j, \arg \max_a Q(S_j, a)) - Q(S_{j-1}, A_{j-1})$ 
12:      Update transition priority  $p_j \leftarrow |\delta_j|$ 
13:      Accumulate weight-change  $\Delta \leftarrow \Delta + w_j \cdot \delta_j \cdot \nabla_\theta Q(S_{j-1}, A_{j-1})$ 
14:    end for
15:    Update weights  $\theta \leftarrow \theta + \eta \cdot \Delta$ , reset  $\Delta = 0$ 
16:    From time to time copy weights into target network  $\theta_{\text{target}} \leftarrow \theta$ 
17:  end if
18:  Choose action  $A_t \sim \pi_\theta(S_t)$ 
19: end for
```

It led to both **faster learning** and **better final policy quality** across most Atari games

Weakness of Experience Replay

- Q: What is the weakness of experience replay?
- A: On-policy methods can not use experience replay, such as SARSA, REINFORCE, A2C, ...

- 1 Experience Replay
- 2 Prioritized ER
- 3 Target Network & Double DQN**
- 4 Dueling Network

Recap: TD Learning for DQN

- Use a transition, s_t, a_t, r_t, s_{t+1} , to update $\mathbf{w}$
- TD target: $y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w})$
- TD target y_t is partly an estimate made by the DQN Q
- TD error: $\delta_t = Q(s_t, a_t; \mathbf{w}) - y_t$
- SGD: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \delta_t \cdot \frac{\partial Q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}}$
- Rewrite SGD: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot (Q(s_t, a_t; \mathbf{w}) - y_t) \cdot \frac{\partial Q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}}$
- We use y_t , which is partly based on Q , to update Q itself.

Problem of Overestimation

- TD learning makes DQN overestimate action-values. (Q: Why?)
- Reason 1: The maximization bias
 - TD target: $y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w})$
 - TD target is bigger than the real action-value
- Reason 2: Bootstrapping propagates the overestimation

Recap: Maximization Bias (Lecture 2.2)

- We often need to maximize over our value estimates. The estimated maxima suffer from **maximization bias**
- Consider a state for which all action $Q_*(s, a) = 0$. Our estimates $Q(s, a)$ are uncertain, some are positive and some negative
- Another intuitive understanding (Jensens Inequality):

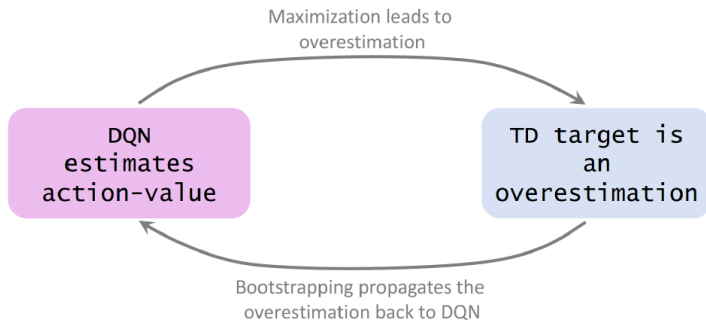
$$\mathbb{E} \left[\max_i \mu_i \right] \geq \max \mathbb{E} [\mu_i]$$

- This is because we use the same estimate Q both to choose the argmax and to evaluate it

Maximization Bias & Bootstrapping

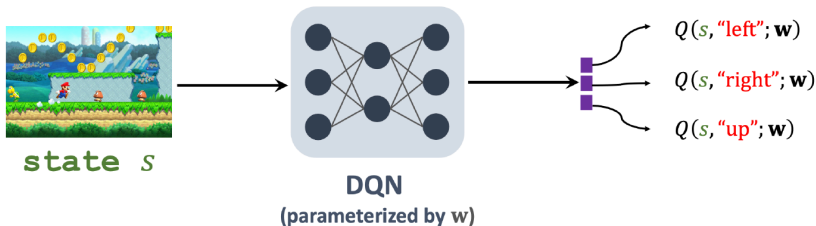
- True action-value: $Q_{\pi}(s, a_1), \dots, Q_{\pi}(s, a_n)$
- Noisy estimations made by DQN: $Q(s, a_1; \mathbf{w}), \dots, Q(s, a_n; \mathbf{w})$
- We conclude that $Q_{t+1} = \max_a Q(s_{t+1}, a; \mathbf{w})$ is an overestimation of the true action-value at time $t+1$
- The TD target, $y_t = r_t + \gamma \cdot Q_{t+1}$, is thereby an overestimation
- When Q_{t+1} is used for updating $Q(s_t, a_t; \mathbf{w})$, the overestimation is propagated back to DQN

Why does overestimation happen?



Why is overestimation harmful?

- The agent is controlled by the DQN: $a_t = \arg\max_a Q(s_t, a; \mathbf{w})$
- Uniform overestimation is not a problem



Why is overestimation harmful?

- The agent is controlled by the DQN: $a_t = \operatorname{argmax}_a Q(s_t, a; \mathbf{w})$
- Uniform overestimation is not a problem

For example,

- $Q_*(s, a_1) = 200, Q_*(s, a_2) = 100, Q_*(s, a_3) = 230$
- Action a_3 will be selected
- Suppose $Q(s, a_i; \mathbf{w}) = Q_*(s, a_i) + 100$, for all a_i
- Then DQN believes a_3 has the highest value and will select a_3

Why is overestimation harmful?

- The agent is controlled by the DQN: $a_t = \operatorname{argmax}_a Q(s_t, a; \mathbf{w})$
- Uniform overestimation is not a problem
- Non-uniform overestimation is problematic

For example,

- $Q_*(s, a_1) = 200, Q_*(s, a_2) = 100, Q_*(s, a_3) = 230$
- $Q(s, a_1; \mathbf{w}) = 280, Q(s, a_2; \mathbf{w}) = 300, Q(s, a_3; \mathbf{w}) = 240$
- Action a_2 will be selected

Why is overestimation harmful?

- Unfortunately, the overestimation is non-uniform
- Q: Why?
- Use a transition, (s_t, a_t, r_t, s_{t+1}) , to update $\mathbf{w}$
- The TD target, y_t , **overestimates** $Q_*(s_t, a_t)$
- TD algorithm pushes $Q(s_t, a_t; \mathbf{w})$ towards y_t
- Thus, $Q(s_t, a_t; \mathbf{w})$ **overestimates** $Q_*(s_t, a_t)$
- The more frequently (s, a) appear in the replay buffer, the worse $Q(s, a; \mathbf{w})$ overestimate $Q_*(s, a)$

Solution

- **Problem:** DQN trained by TD overestimates action-values
- **Solution 1:** Use a target network [1] to compute TD targets (address the problem caused by bootstrapping)
- **Solution 2:** Use double DQN [2] to alleviate the overestimation caused by maximization.

[1] Human-level Control Through Deep Reinforcement Learning. Nature, 2015.

[2] Deep Reinforcement Learning with Double Q-Learning. In AAAI, 2016.

Target Network

- **Target network:** $Q(s, a; \mathbf{w}^-)$
 - The same network structure as the DQN, $Q(s, a; \mathbf{w})$
 - Different parameters: $\mathbf{w}^- = \mathbf{w}$
- Use $Q(s, a; \mathbf{w})$ to **control the agent** and **collect experience**:

$$(s_t, a_t, r_t, s_{t+1})$$

- Use $Q(s, a; \mathbf{w}^-)$ to **compute TD target**

$$y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}^-)$$

TD Learning with Target Network

TD Learning with Target Network

- Use a transition, (s_t, a_t, r_t, s_{t+1}) , to update $\mathbf{w}$
- TD target: $y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}^-)$
- TD error: $\delta_t = Q(s_t, a_t; \mathbf{w}) - y_t$
- SGD: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \delta_t \cdot \frac{\partial Q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}}$
- **Update Target Network:** Periodically update $\mathbf{w}^-$
 - Option 1: $\mathbf{w}^- \leftarrow \mathbf{w}$
 - Option 2: $\mathbf{w}^- \leftarrow \tau \cdot \mathbf{w} + (1 - \tau) \cdot \mathbf{w}^-$

Deep Q-Network

So far,

- Experience replay
- Target network: we used an iterative update that adjusts the action-values (Q) towards target values that are only periodically updated, thereby reducing correlations with the target.
- **Results:**
 - Outperforms the best existing RL methods on 43 of 49 games
 - It was comparable to that of professional human games tester across 49 games, achieving more than 75% of human score on half of the games

Human-level Control through Deep Reinforcement Learning. Nature, 2015.

Algorithm

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

 With probability ε select a random action a_t

 otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

 Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

 Every C steps reset $\hat{Q} = Q$

End For

End For

Human-level Control through Deep Reinforcement Learning. Nature, 2015.

Recap: Double Q-Learning (Lecture 2.2)

- Train **two action-value** functions, Q_1 and Q_2
- Do Q-learning on both, but
 - Never on the same time steps (Q_1 and Q_2 are independent)
 - Pick Q_1 or Q_2 **at random** to be updated on each step
- If updating Q_1 , use Q_2 for the value of the **next state**:

$$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha \left(r_t + Q_2 \left(s_{t+1}, \arg \max_a Q_1(s_{t+1}, a) \right) - Q_1(s_t, a_t) \right)$$

- **Inference:** Action selections are ϵ -greedy with respect to the sum of Q_1 and Q_2

Recap: Double Tabular Q-Learning (Lecture 2.2)

Initialize $Q_1(s, a)$ and $Q_2(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily
Initialize $Q_1(\text{terminal-state}, \cdot) = Q_2(\text{terminal-state}, \cdot) = 0$
Repeat (for each episode):
 Initialize S
 Repeat (for each step of episode):
 Choose A from S using policy derived from Q_1 and Q_2 (e.g., ϵ -greedy in $Q_1 + Q_2$)
 Take action A , observe R, S'
 With 0.5 probability:
 $Q_1(S, A) \leftarrow Q_1(S, A) + \alpha(R + \gamma Q_2(S', \arg\max_a Q_1(S', a)) - Q_1(S, A))$
 else:
 $Q_2(S, A) \leftarrow Q_2(S, A) + \alpha(R + \gamma Q_1(S', \arg\max_a Q_2(S', a)) - Q_2(S, A))$
 $S \leftarrow S'$;
 until S is terminal

Double-Q estimation $\leq \max_i \mathbb{E}[X_i] \leq \mathbb{E}[\max_i X_i]$ (single-Q estimation)

Double Q-learning. In NIPS, 2010.

Double DQN

Double DQN

- Action selection using the DQN:

$$a^* = \arg \max_a Q(s_{t+1}, a; \mathbf{w})$$

- Evaluation using **target network**:

$$y_t = r_t + \gamma \cdot Q(s_{t+1}, a^*; \mathbf{w}^-)$$

- It works well in most cases, but overestimation still may happen

Deep Reinforcement Learning with Double Q-learning. In AAAI, 2016.

Summary: Overestimation

Summary:

- Because of the **maximization**, the TD target overestimates the true action-value
- By creating a “positive feedback loop”, **bootstrapping** further exacerbates the overestimation.
- **Target network** can partly avoid bootstrapping. (Not completely, because w^- depends on w .)
- **Double DQN** alleviates the overestimation caused by the maximization.

- ① Experience Replay
- ② Prioritized ER
- ③ Target Network & Double DQN
- ④ Dueling Network

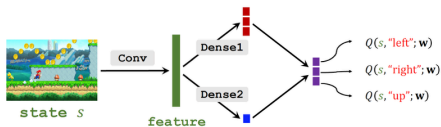
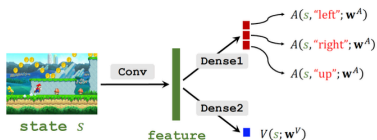
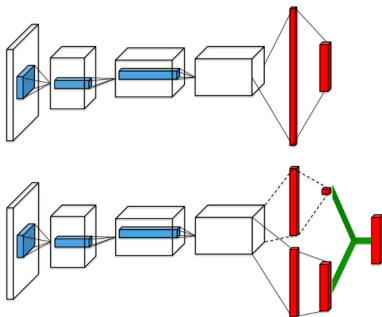
Advantage Function

- Action-value function: $Q_{\pi}(s_t, a_t) = \mathbb{E}[G_t | s_t, a_t]$
- State-value function: $V_{\pi}(s_t) = \mathbb{E}_a[Q_{\pi}(s_t, a)]$
- Advantage function: $A_{\pi}(s_t, a_t) = Q_{\pi}(s_t, a_t) - V_{\pi}(s_t)$
- Optimal action-value function: $Q_*(s, a) = \max_{\pi} Q_{\pi}(s, a)$
- Optimal state-value function: $V_*(s) = \max_{\pi} V_{\pi}(s)$
- Optimal advantage function: $A_*(s, a) = Q_*(s, a) - V_*(s)$
- Idea borrowed from Policy Gradient (will be introduced in the following lectures)

Dueling Network Architectures for Deep Reinforcement Learning. In ICML, 2016.

Dueling Network

Dueling Architecture: Two-stream Q network (vs. single-stream Q network)



Dueling Network Architectures for Deep Reinforcement Learning. In ICML, 2016.

Properties of Advantage Function

- Theorem 1: $V_*(s) = \max_a Q_*(s, a)$
- Recall the definition of the optimal advantage function

$$A_*(s, a) = Q_*(s, a) - V_*(s)$$

- It follows that

$$\max_a A_*(s, a) = \max_a Q_*(s, a) - V_*(s) = 0$$

Dueling Network Architectures for Deep Reinforcement Learning. In ICML, 2016.

Properties of Advantage Function

- Definition of advantage:

$$A_*(s, a) = Q_*(s, a) - V_*(s, a)$$



-

$$Q_*(s, a) = V_*(s) + A_*(s, a)$$



-

$$Q_*(s, a) = V_*(s) + A_*(s, a) - \max_a A_*(s, a)$$

Dueling Network Architectures for Deep Reinforcement Learning. In ICML, 2016.

Dueling Network: Formulation

- **Theorem 2:** $Q_*(s, a) = V_*(s) + A_*(s, a) - \max_a A_*(s, a)$
- Approximate $V_*(s)$ by a neural network, $V(s; \mathbf{w}^V)$
- Approximate $A_*(s, a)$ by a neural network, $A_*(s, a; \mathbf{w}^A)$
- Thus, approximate $Q_*(s, a)$ by the dueling network

$$Q(s, a; \mathbf{w}^A, \mathbf{w}^V) = V(s; \mathbf{w}^V) + A(s, a; \mathbf{w}^A) - \max_a A(s, a; \mathbf{w}^A)$$

$\mathbf{w} = (\mathbf{w}^A, \mathbf{w}^V)$ are the learnable parameters

Training

- Dueling network, $Q(s, a; \mathbf{w})$, is an approximation to $Q_*(s, a)$
- Learn the parameter, $\mathbf{w} = (\mathbf{w}^A, \mathbf{w}^V)$, in the same way as the other DQNs
- Previous tricks can be used in the same way:
 - Prioritized experience replay
 - Double DQN
 - Multi-step TD target

Dueling Network Architectures for Deep Reinforcement Learning. In ICML, 2016.

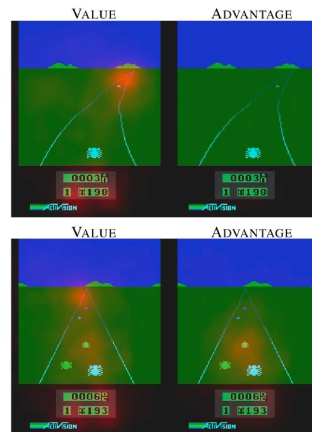
Overcome Non-identifiability

- Equation 1: $Q_*(s, a) = V_*(s) + A_*(s, a)$
- Equation 2: $Q_*(s, a) = V_*(s) + A_*(s, a) - \max_a A_*(s, a)$ Question: why it is the zero term necessary?
- Equation 1 has the problem of non-identifiability.
 - Let $V' = V_* + 10$ and $A' = A_* - 10$
 - Then $Q_*(s, a) = V_*(s) + A_*(s, a) = V'(s) + A'(s, a)$
- An alternative variant replaces the max operator with an average
 - $Q_*(s, a) = V_*(s, a) + A_*(s, a) - \text{mean}_a A_*(s, a)$
- It empirically increases the stability of the optimization

Dueling Network Architectures for Deep Reinforcement Learning. In ICML, 2016.

Dueling Network

- Value function + Advantage function
- The **value stream** learns to pay attention to the road.
- The **advantage stream** learns to pay attention only when there are cars immediately in front, so as to avoid collisions.



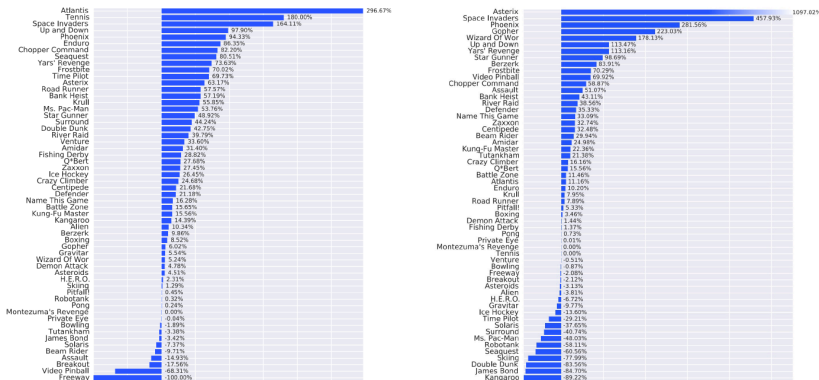
Dueling Network Architectures for Deep Reinforcement Learning. In ICML, 2016.

Advantages of Dueling Network

- **Improved efficiency.** It effectively learns the value of states and the relative advantages of actions, which helps in scenarios **where many actions have similar values**.
- **Improved stability.** It helps to stabilize training by allowing the model to learn the value of states even **when some actions are rarely chosen**. It is useful in environments with many actions, as it prevents the network from overfitting to specific actions.
- **Better generalization.** By focusing on the state value, it generalizes better across similar states, improving performance in environments where certain actions are rarely taken but can still be crucial in specific states.

Dueling Network

Improvements of dueling architecture over the baseline Single network



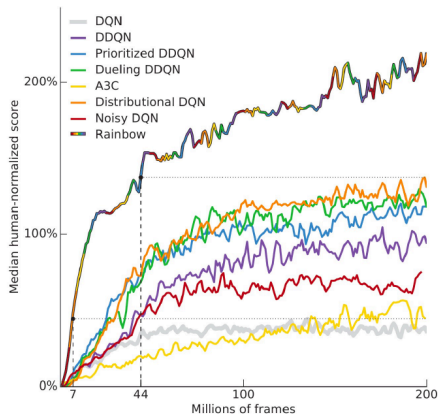
Dueling Network Architectures for Deep Reinforcement Learning. In ICML, 2016.

Rainbow: Combining All Tricks

- Multi-step bootstrap targets
- Maximum bias (Double DQN)
- Prioritized experience replay
- Dueling network architecture
- Noisy DQN
- Distributional Q-Learning

Rainbow: Combining Improvements in Deep Reinforcement Learning. In AAAI, 2018.

Rainbow: Combining All Tricks



Rainbow: Combining Improvements in Deep Reinforcement Learning. In AAAI, 2018.

Extra Reading Materials

- Double Q-learning. In NIPS, 2010.
- Playing Atari with Deep Reinforcement Learning. In NeurIPS workshop, 2013.
- Human-level Control through Deep Reinforcement Learning. Nature, 2015.
- Deep Reinforcement Learning with Double Q-learning. In AAAI, 2016.
- Prioritized Experience Replay. In ICLR, 2016.
- Dueling Network Architectures for Deep Reinforcement Learning. In ICML, 2016.
- A Distributional Perspective on Reinforcement Learning. In ICML, 2017.
- Noisy Networks for Exploration. In ICLR, 2018.
- Rainbow: Combining Improvements in Deep Reinforcement Learning. In AAAI, 2018.